

TZAR-BOT

Penetration-Testing Automation Platform

Architecture & Internal Structure

Architecture & Internal Structure

A technical reference for how the platform works and how it is built

Version 0.1.0 · June 2026

Authorized security testing use only

1. Executive Overview

Tzar-Bot is an AI-driven penetration-testing automation platform. It turns a natural-language engagement request — a target, its scope, and the rules of engagement — into a structured, evidence-backed security assessment carried out by a team of cooperating AI agents, and produces a validated report at the end.

What distinguishes Tzar-Bot from “an LLM with a shell” is its central design principle: the artificial intelligence makes the judgement calls, but deterministic code enforces the safety boundaries. The model decides what to test and how; the harness decides what is actually allowed to run. This division is what makes an autonomous offensive tool trustworthy.

The platform is built around three agent roles — a Coordinator that plans and delegates, Executors that perform the gated testing, and Validators that adversarially confirm each finding — supported by a library of 67 expert-methodology skills, 25 stdlib-first command-line tools, two MCP servers, and a code-enforced scope engine wired into every action.

2. Core Design Principle — “AI decides, code enforces”

Every capability in Tzar-Bot is split along the same line:

- **The AI provides judgement** — what to test, which vulnerability class fits, how to chain findings, how to write the report. This is open-ended reasoning, where the model excels and where rigid rules would fail.
- **Code provides the guardrails** — scope enforcement, write containment, finding validation, rate limiting, and budget ceilings. These are deterministic and non-negotiable, so a creative (or compromised) model cannot step outside them.

Concretely: when an agent wants to run a scanner, it does not touch a socket. It emits a structured tool request; the harness intercepts it, runs it through the scope engine and the other gates, and only then executes it. The model proposes; the harness disposes.

3. High-Level Architecture

An engagement flows top-to-bottom through the layers below. The runtime owns the agent loop; the Coordinator plans and delegates; Executors do the gated work; Validators confirm; a report is generated. The code-enforced gate (right) sits across the action band: every tool call passes through it before it runs.

Figure 1 — High-Level Architecture

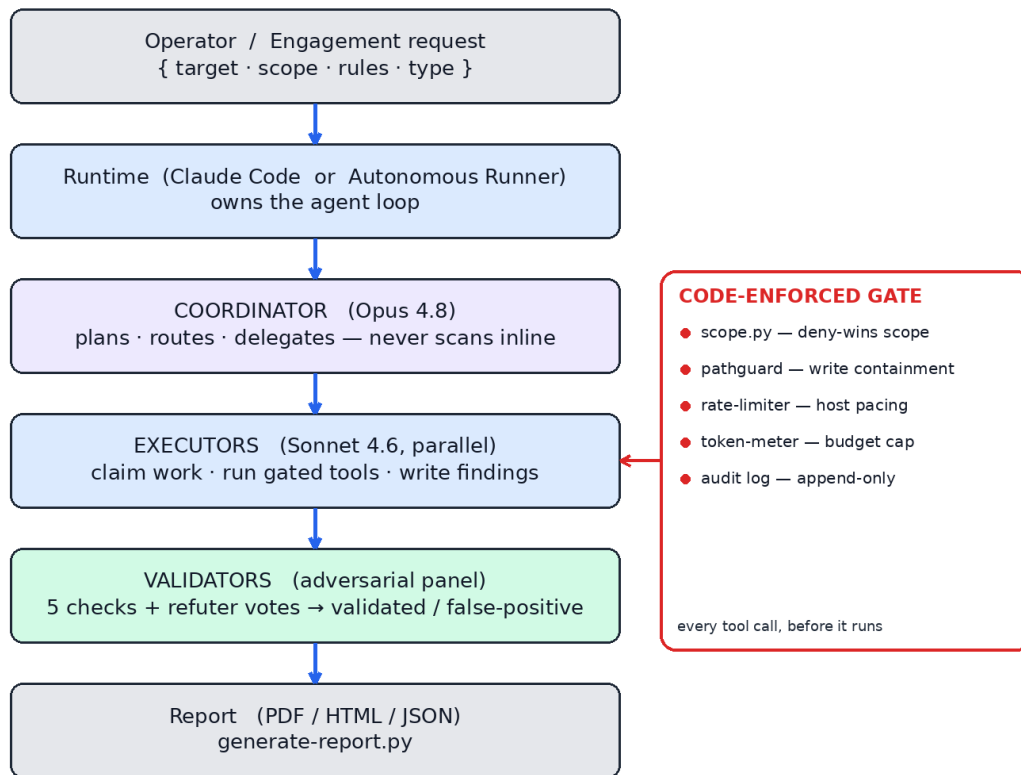


Figure 1 — High-level architecture and the code-enforced gate.

4. The Agent Model

Tzar-Bot uses a three-role agent triangle. Roles are separated so that planning, execution, and verification never collapse into a single context that could rubber-stamp its own work.

Figure 2 — Coordinator / Executor / Validator

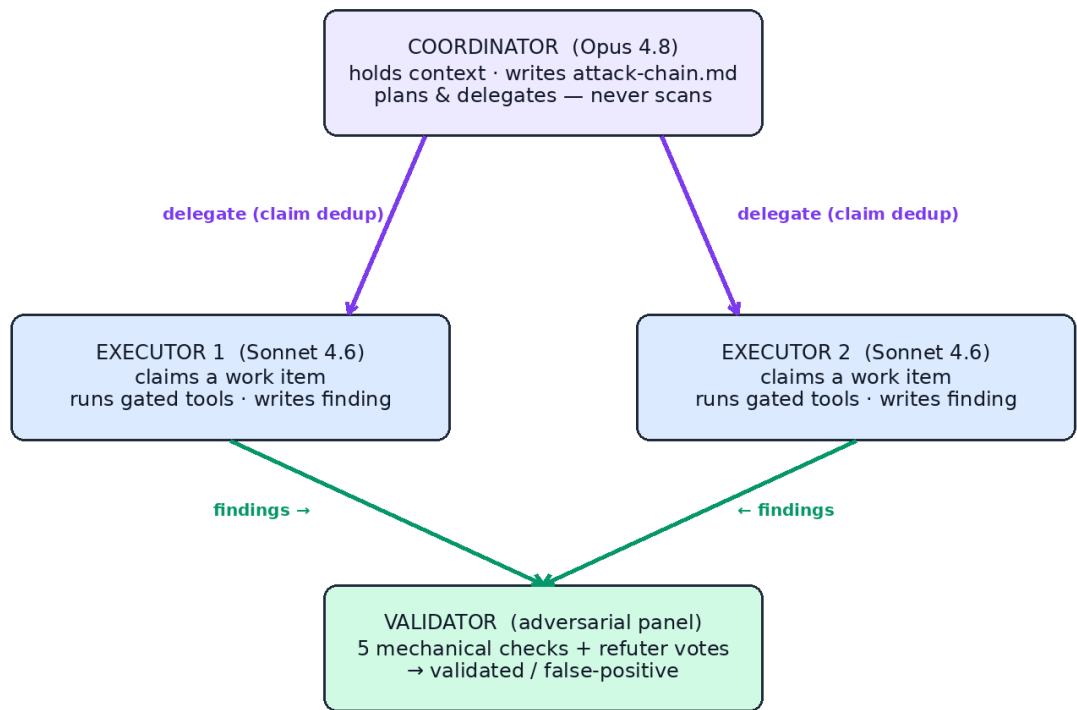


Figure 2 — The Coordinator / Executor / Validator triangle.

Role	Model	Responsibility
Coordinator	Opus 4.8	Holds engagement context; writes reasoning to attack-chain.md; plans and delegates. Never runs scanning tools itself.
Executor	Sonnet 4.6	Receives one focused work item, claims it, runs the gated tools, and writes findings with evidence. Multiple run in parallel.
Validator	Sonnet 4.6	Adversarially confirms each finding: a mechanical 5-check pre-check plus a panel of independent refuters voting from distinct lenses.

The Coordinator hard-boundary — “never scan inline” — is enforced structurally by simply not giving the Coordinator the scanning tools. It can only plan and delegate.

5. Code-Enforced Safety — The Tool Gate

The tool gate is the single interception point that makes autonomy safe. Every tool call an agent emits passes through it, in order, before anything executes.

Figure 3 — The Tool Gate (every model tool call)

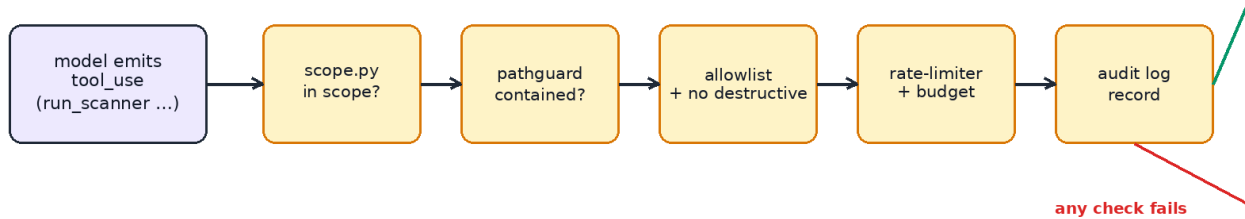


Figure 3 — The tool gate every model tool call must pass.

Stage	Control	Effect
1. Scope	scope.py	Deny-wins, default-deny. Out-of-scope host → denied.
2. Path	pathguard	Writes contained to the engagement sandbox.
3. Allowlist	scanner allow/denylist	Only approved scanners; destructive patterns blocked.
4. Pace / budget	rate-limiter + token-meter	Per-host throttle; hard output-token ceiling.
5. Audit	append-only audit.log	Every call and decision recorded.

A denied call is handed back to the model as an error result, so it re-plans — the engagement continues safely rather than breaking. The same scope authority (scope.py) backs the Bash PreToolUse hook, the browser MCP server, and the autonomous runner, so there is one source of truth.

6. Internal Structure

The platform is organised into tools, skills, MCP servers, state/memory, typed output folders, and a governance layer.

Figure 5 — Internal Structure

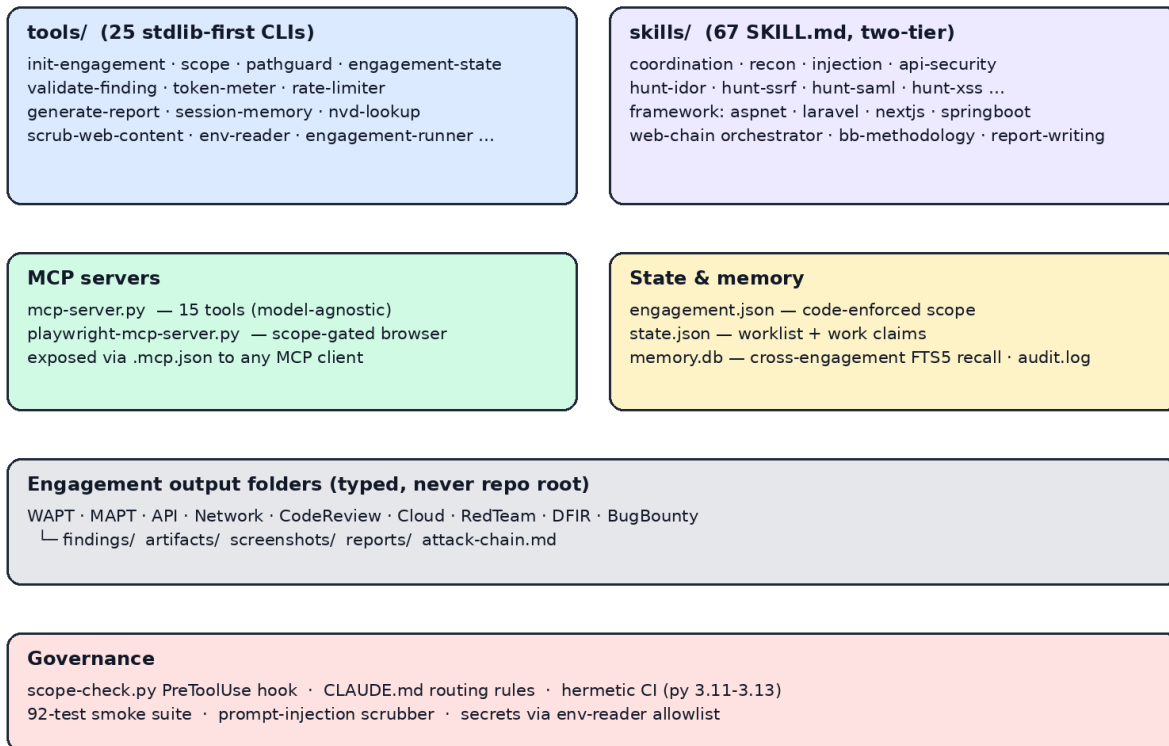


Figure 5 — Internal component map.

6.1 Tools

Twenty-five stdlib-first Python command-line utilities. The core (engagement init, scope, state, validation, reporting, the MCP servers) runs on the Python standard library with zero third-party dependencies; optional feature tools (PDF, DOCX, browser, the autonomous runner) declare their dependencies as packaging extras.

6.2 Skills

Sixty-seven expert-methodology skill files in a two-tier structure: a broad skill acts as a quick-start and router, while deep per-class material lives in hunt-* references. The skills encode professional pentest methodology and are the platform's real domain depth — they convert a general model into a specialist.

6.3 State & Memory

- **engagement.json** — the code-enforced scope for the engagement.
- **state.json** — the resumable worklist plus work-claims so executors don't double-test a surface.
- **memory.db** — cross-engagement full-text recall (SQLite FTS5).
- **audit.log** — the append-only record of every gated action.

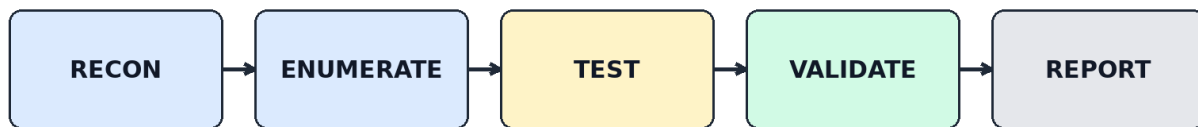
6.4 Output Routing

Every engagement writes into a typed, project-named folder (WAPT, MAPT, API, Network, CodeReview, Cloud, RedTeam, DFIR, BugBounty) — never the repository root. Inside it live findings/, artifacts/, screenshots/, reports/, and attack-chain.md. This routing rule is enforced in code by the path-containment guard.

7. Engagement Lifecycle

An engagement advances through five phases. Phase, worklist, and claims are persisted, so a long engagement is resumable after interruption and every step is scope-guarded.

Figure 6 — Engagement Lifecycle



resumable: phase + worklist + claims persisted in state.json; every step scope-guarded

Figure 6 — The five engagement phases.

8. The Autonomous Engagement Runner

The autonomous runner is the component that lets Tzar-Bot run an entire engagement unattended. It is a manual agentic loop on the Claude API: the platform owns the loop, so every tool call is gated before it executes. The Coordinator plans and delegates; each delegation spawns an Executor that claims its item, tests it under the gate, and reports back; findings are then adversarially validated.

Figure 4 — Autonomous Engagement Runner — control loop

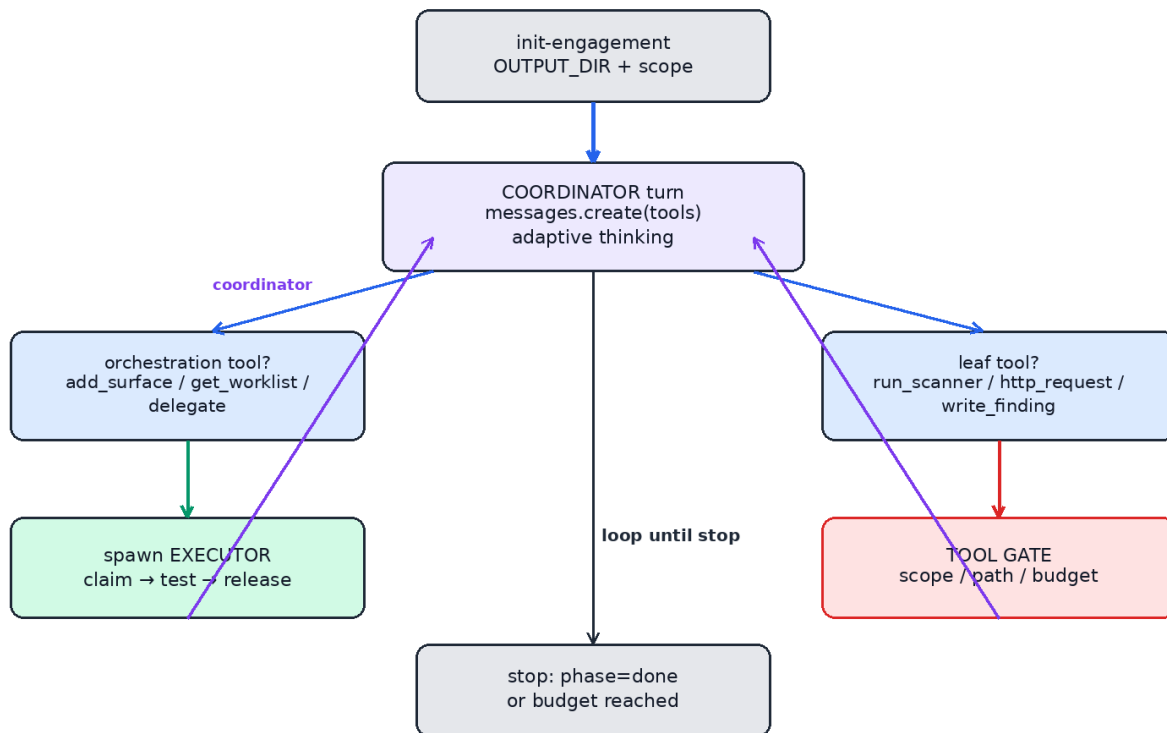


Figure 4 — The autonomous runner control loop.

Cost is bounded and measured: the system prompt (the mounted skill) is prompt-cached for cheap reads, and the token-meter enforces a hard budget ceiling. The trust-critical behaviours — staying in scope, never double-testing a surface, and confirming findings adversarially — are all enforced in code and covered by an offline self-test.

9. Key Tooling Reference

Tool	Purpose
init-engagement.py	Create the typed OUTPUT_DIR, scope, and engagement.json.
scope.py	Code-enforced scope authority (apex/wildcard/CIDR/regex, deny-wins).
pathguard.py	Contain file writes to the engagement sandbox.
engagement-state.py	Resumable worklist + executor work-claim ledger.
validate-finding.py	Mechanical 5-check validation protocol.
token-meter.py	Per-agent token/cost telemetry and budgets.
rate-limiter.py	Per-host token-bucket request pacing.
scrub-web-content.py	Strip prompt-injection patterns from web content.
env-reader.py	The only approved, allow-listed way to read secrets.

generate-report.py	Produce the final PDF report.
engagement-runner.py	The autonomous coordinator/executor/validator loop.

10. Security Posture & Hardening

Because Tzar-Bot is itself a security tool, its own safety is treated as a first-class concern:

- **Code-enforced scope** — deny-wins / default-deny, applied to Bash commands, the browser MCP server, and the autonomous runner from one shared module.
- **Secrets hygiene** — credentials are read only through an allow-listed reader; .env files and databases are never committed.
- **Prompt-injection defence** — web-sourced content is scrubbed before it enters an agent prompt.
- **MCP hardening** — the MCP servers enforce scope on browser navigation, contain all writes, time out subprocesses, survive malformed input, and reject argument injection.
- **Hermetic CI** — a 92-test smoke suite runs across Python 3.11–3.13 with no network, plus scope self-tests and skill linting.

11. Maturity & Direction

Tzar-Bot today is a strong, code-safe framework with the autonomous agent triangle implemented and self-tested. The trust-critical core — staying in scope, no double-work, adversarially validated findings, and a coordinator that structurally cannot scan — is in place. Ongoing direction includes checkpoint/resume for long runs, graceful budget wind-down, and the broader product layer (packaging, multi-tenant isolation, and a control plane) for a hosted offering.

— End of document —